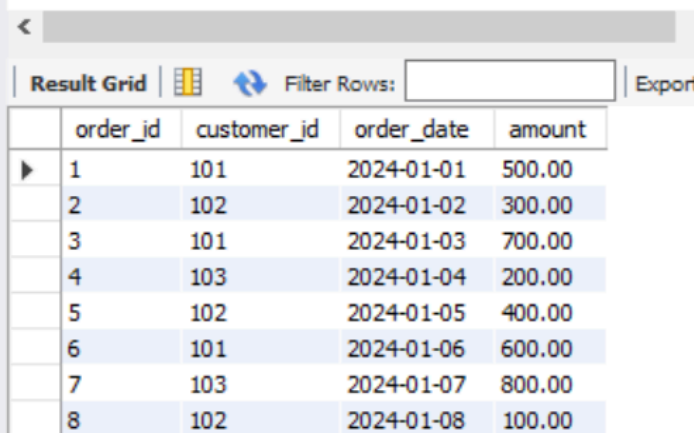


```
CREATE TABLE orders (  
  order_id INT,  
  customer_id INT,  
  order_date DATE,  
  amount DECIMAL(10,2)  
);
```

```
INSERT INTO orders VALUES  
(1, 101, '2024-01-01', 500),  
(2, 102, '2024-01-02', 300),  
(3, 101, '2024-01-03', 700),  
(4, 103, '2024-01-04', 200),  
(5, 102, '2024-01-05', 400),  
(6, 101, '2024-01-06', 600),  
(7, 103, '2024-01-07', 800),  
(8, 102, '2024-01-08', 100);
```

```
1 • select * from orders;
```



The screenshot shows a database interface with a query result grid. The grid has 5 columns: order_id, customer_id, order_date, and amount. There are 8 rows of data. The interface includes a 'Result Grid' tab, a 'Filter Rows' input field, and an 'Export' button.

	order_id	customer_id	order_date	amount
▶	1	101	2024-01-01	500.00
	2	102	2024-01-02	300.00
	3	101	2024-01-03	700.00
	4	103	2024-01-04	200.00
	5	102	2024-01-05	400.00
	6	101	2024-01-06	600.00
	7	103	2024-01-07	800.00
	8	102	2024-01-08	100.00

Questions

1. For each order, show the total number of orders placed by that customer.

```
SELECT *, COUNT(*) OVER (PARTITION BY customer_id) AS total_orders  
  
FROM orders;
```

```

3 • select customer_id, count(order_id)
4   from orders
5  group by customer_id;

```

Result Grid		Filter Rows:	Export:
	customer_id	count(order_id)	
▶	101	3	
	102	3	
	103	2	

Ideal and correct only-

```

3 • select *, count(*) over (partition by customer_id) as total_orders
4   from orders;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
order_id	customer_id	order_date	amount	total_orders
1	101	2024-01-01	500.00	3
3	101	2024-01-03	700.00	3
6	101	2024-01-06	600.00	3
2	102	2024-01-02	300.00	3
5	102	2024-01-05	400.00	3
8	102	2024-01-08	100.00	3
4	103	2024-01-04	200.00	2
7	103	2024-01-07	800.00	2

2. Display each order along with cumulative total amount spent by that customer (based on order date).

SELECT *, SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)

FROM orders;

```

3 • select *, sum(amount) over (partition by customer_id order by order_date)
4   from orders;

```

< Result Grid Filter Rows: Export: Wrap Cell Content:					
	order_id	customer_id	order_date	amount	sum(amount) over (partition by customer_id order by order_date)
▶	1	101	2024-01-01	500.00	500.00
	3	101	2024-01-03	700.00	1200.00
	6	101	2024-01-06	600.00	1800.00
	2	102	2024-01-02	300.00	300.00
	5	102	2024-01-05	400.00	700.00
	8	102	2024-01-08	100.00	800.00
	4	103	2024-01-04	200.00	200.00
	7	103	2024-01-07	800.00	1000.00

Better version-

SUM(amount) OVER (

PARTITION BY customer_id

ORDER BY order_date

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

)

👉 Why?

Avoids ambiguity with duplicate dates

3. Show the previous order amount for each customer.

SELECT *, LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date)

FROM orders;

```

3 • select *, lag(amount) over (partition by customer_id order by order_date)
4   from orders;

```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	order_id	customer_id	order_date	amount	lag(amount) over (partition by customer_id order by order_date)
▶	1	101	2024-01-01	500.00	NULL
	3	101	2024-01-03	700.00	500.00
	6	101	2024-01-06	600.00	700.00
	2	102	2024-01-02	300.00	NULL
	5	102	2024-01-05	400.00	300.00
	8	102	2024-01-08	100.00	400.00
	4	103	2024-01-04	200.00	NULL
	7	103	2024-01-07	800.00	200.00

- For each order, show the difference between current and previous order amount for that customer.

SELECT *,

amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS
diff

FROM orders;

(If you use ABS(), you'll lose direction like + & - signs indicating increase and decrease)

```

3 • select *, abs(amount - lag(amount) over (partition by customer_id order by order_date)) as diff
4   from orders;

```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	order_id	customer_id	order_date	amount	diff
▶	1	101	2024-01-01	500.00	NULL
	3	101	2024-01-03	700.00	200.00
	6	101	2024-01-06	600.00	100.00
	2	102	2024-01-02	300.00	NULL
	5	102	2024-01-05	400.00	100.00
	8	102	2024-01-08	100.00	300.00
	4	103	2024-01-04	200.00	NULL
	7	103	2024-01-07	800.00	600.00

- Assign a rank to orders based on amount (highest first).

SELECT *, RANK() OVER (ORDER BY amount DESC)

FROM orders;

```
3 • select *, rank() over (order by amount desc)
4   from orders;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	order_id	customer_id	order_date	amount	rank() over (order by amount desc)
▶	7	103	2024-01-07	800.00	1
	3	101	2024-01-03	700.00	2
	6	101	2024-01-06	600.00	3
	1	101	2024-01-01	500.00	4
	5	102	2024-01-05	400.00	5
	2	102	2024-01-02	300.00	6
	4	103	2024-01-04	200.00	7
	8	102	2024-01-08	100.00	8

6. Assign rank to orders **within each customer** based on amount.

SELECT *, RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC)

FROM orders;

```
3 • select *, rank() over (partition by customer_id order by amount desc)
4   from orders;
```

Result Grid  Filter Rows: Export:  Wrap Cell Content: 					
	order_id	customer_id	order_date	amount	rank() over (partition by customer_id order by amount desc)
▶	3	101	2024-01-03	700.00	1
	6	101	2024-01-06	600.00	2
	1	101	2024-01-01	500.00	3
	5	102	2024-01-05	400.00	1
	2	102	2024-01-02	300.00	2
	8	102	2024-01-08	100.00	3
	7	103	2024-01-07	800.00	1
	4	103	2024-01-04	200.00	2

7. Show the highest order amount for each customer in every row.

```
SELECT *, MAX(amount) OVER (PARTITION BY customer_id) AS highest_order  
FROM orders;
```

```
3 • select *, max(amount) over (partition by customer_id) as highest_order  
4 from orders;
```

	order_id	customer_id	order_date	amount	highest_order
▶	1	101	2024-01-01	500.00	700.00
	3	101	2024-01-03	700.00	700.00
	6	101	2024-01-06	600.00	700.00
	2	102	2024-01-02	300.00	400.00
	5	102	2024-01-05	400.00	400.00
	8	102	2024-01-08	100.00	400.00
	4	103	2024-01-04	200.00	800.00
	7	103	2024-01-07	800.00	800.00

✖ SCENARIO 2: Employee Salaries

```
CREATE TABLE employees (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    dept_id INT,  
    salary INT  
);
```

```
INSERT INTO employees VALUES  
(1, 'Alice', 10, 5000),  
(2, 'Bob', 10, 6000),  
(3, 'Charlie', 10, 6000),  
(4, 'David', 20, 4000),  
(5, 'Eva', 20, 7000),  
(6, 'Frank', 20, 3000),  
(7, 'Grace', 30, 8000),  
(8, 'Helen', 30, 8000);
```

```
1 • select * from employees;
2
```

	emp_id	emp_name	dept_id	salary
▶	1	Alice	10	5000
	2	Bob	10	6000
	3	Charlie	10	6000
	4	David	20	4000
	5	Eva	20	7000
	6	Frank	20	3000
	7	Grace	30	8000
	8	Helen	30	8000

Questions

1. Show each employee along with the highest salary in their department.

```
SELECT *, MAX(salary) OVER (PARTITION BY dept_id) AS max_salary_dept
FROM employees;
```

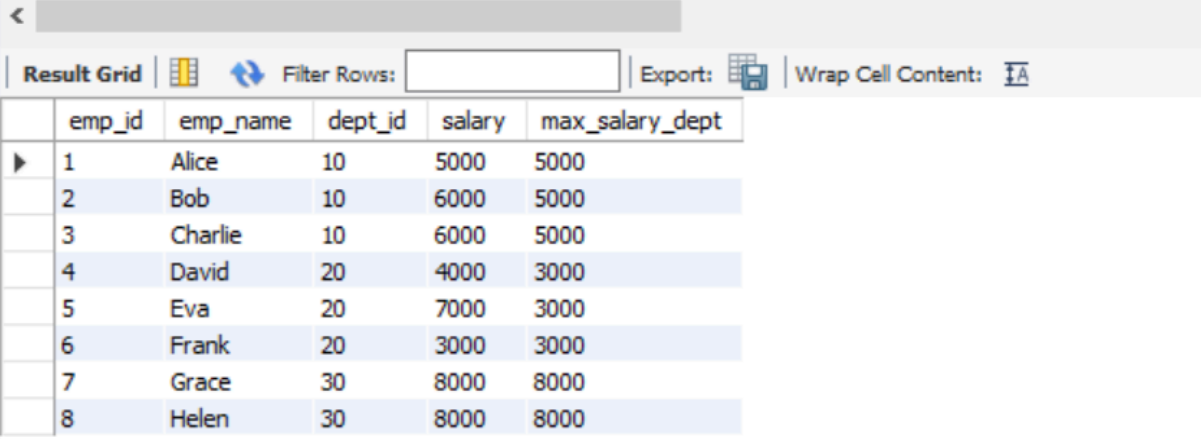
```
3 • select *, max(salary) over (partition by dept_id) as max_salary_dept
4 from employees;
```

	emp_id	emp_name	dept_id	salary	max_salary_dept
▶	1	Alice	10	5000	6000
	2	Bob	10	6000	6000
	3	Charlie	10	6000	6000
	4	David	20	4000	7000
	5	Eva	20	7000	7000
	6	Frank	20	3000	7000
	7	Grace	30	8000	8000
	8	Helen	30	8000	8000

2. Show each employee along with the lowest salary in their department.

```
SELECT *, MIN(salary) OVER (PARTITION BY dept_id) AS min_salary_dept  
FROM employees;
```

```
3 • select *, min(salary) over (partition by dept_id) as max_salary_dept  
4 from employees;
```



	emp_id	emp_name	dept_id	salary	max_salary_dept
▶	1	Alice	10	5000	5000
	2	Bob	10	6000	5000
	3	Charlie	10	6000	5000
	4	David	20	4000	3000
	5	Eva	20	7000	3000
	6	Frank	20	3000	3000
	7	Grace	30	8000	8000
	8	Helen	30	8000	8000

3. Find employees whose salary is greater than the average salary of their department.

```
SELECT *  
FROM (  
    SELECT *, AVG(salary) OVER (PARTITION BY dept_id) AS avg_salary  
    FROM employees  
) t  
WHERE salary > avg_salary;
```

Normal method-

```
select * from employees e1  
where e1.salary > (  
select avg(e2.salary)
```


from employees e2

where e1.dept_id = e2.dept_id

);

```
3 • select * from employees e1
4   where e1.salary > (
5     select avg(e2.salary)
6     from employees e2
7     where e1.dept_id = e2.dept_id
8   );
```

emp_id	emp_name	dept_id	salary
2	Bob	10	6000
3	Charlie	10	6000
5	Eva	20	7000

4. Assign a rank to employees based on salary within each department.

SELECT *, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC)

FROM employees;

```
3 • select *, rank() over (partition by dept_id order by salary desc)
4   from employees;
```

emp_id	emp_name	dept_id	salary	rank() over (partition by dept_id order by salary desc)
2	Bob	10	6000	1
3	Charlie	10	6000	1
1	Alice	10	5000	3
5	Eva	20	7000	1
4	David	20	4000	2
6	Frank	20	3000	3
7	Grace	30	8000	1
8	Helen	30	8000	1

5. Assign a unique rank (no ties) to employees within each department.

SELECT *, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC)

FROM employees;

```

3 • select *, row_number() over (order by salary desc)
4   from employees;

```

	emp_id	emp_name	dept_id	salary	row_number() over (order by salary desc)
▶	7	Grace	30	8000	1
	8	Helen	30	8000	2
	5	Eva	20	7000	3
	2	Bob	10	6000	4
	3	Charlie	10	6000	5
	1	Alice	10	5000	6
	4	David	20	4000	7
	6	Frank	20	3000	8

6. Show the second highest salary in each department.

```

SELECT *
FROM (
    SELECT *,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2;

```

7. Show employees who earn the same salary as someone else in their department.

```

SELECT *
FROM (
    SELECT *, COUNT(*) OVER (PARTITION BY dept_id, salary) AS cnt
    FROM employees
) t
WHERE cnt > 1;

```

```

2 • SELECT *
3 FROM (
4     SELECT *, COUNT(*) OVER (PARTITION BY dept_id, salary) AS cnt
5     FROM employees
6 ) t
7 WHERE cnt > 1;

```

	emp_id	emp_name	dept_id	salary	cnt
▶	2	Bob	10	6000	2
	3	Charlie	10	6000	2
	7	Grace	30	8000	2
	8	Helen	30	8000	2

The inner query-

```

2 • SELECT *, COUNT(*) OVER (PARTITION BY dept_id, salary) AS cnt
3     FROM employees;

```

	emp_id	emp_name	dept_id	salary	cnt
▶	1	Alice	10	5000	1
	2	Bob	10	6000	2
	3	Charlie	10	6000	2
	6	Frank	20	3000	1
	4	David	20	4000	1
	5	Eva	20	7000	1
	7	Grace	30	8000	2
	8	Helen	30	8000	2

Using JOINS-

SELECT e1.*

FROM employees e1

JOIN employees e2

ON e1.dept_id = e2.dept_id

AND e1.salary = e2.salary

AND e1.emp_id <> e2.emp_id;

SCENARIO 3: Product Sales

```
CREATE TABLE sales (  
    sale_id INT,  
    product_id INT,  
    sale_date DATE,  
    revenue INT  
);
```

```
INSERT INTO sales VALUES  
(1, 101, '2024-01-01', 100),  
(2, 101, '2024-01-02', 150),  
(3, 101, '2024-01-03', 200),  
(4, 102, '2024-01-01', 300),  
(5, 102, '2024-01-02', 250),  
(6, 102, '2024-01-03', 400);
```

Result Grid   Filter Rows:					E
	sale_id	product_id	sale_date	revenue	
▶	1	101	2024-01-01	100	
	2	101	2024-01-02	150	
	3	101	2024-01-03	200	
	4	102	2024-01-01	300	
	5	102	2024-01-02	250	
	6	102	2024-01-03	400	

Questions

1. Show cumulative revenue per product.

```
SELECT *,  
    SUM(revenue) OVER (  
        PARTITION BY product_id  
        ORDER BY sale_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS cumulative_revenue  
FROM sales;
```

```

3 • SELECT *,
4     SUM(revenue) OVER (
5         PARTITION BY product_id
6         ORDER BY sale_date
7         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
8     ) AS cumulative_revenue
9 FROM sales;

```

sale_id	product_id	sale_date	revenue	cumulative_revenue
1	101	2024-01-01	100	100
2	101	2024-01-02	150	250
3	101	2024-01-03	200	450
4	102	2024-01-01	300	300
5	102	2024-01-02	250	550
6	102	2024-01-03	400	950

2. Show the highest revenue per product in each row.

```

SELECT *, MAX(revenue) OVER (PARTITION BY product_id)
FROM sales;

```

```

3 • SELECT *, MAX(revenue) OVER (PARTITION BY product_id)
4 FROM sales;

```

sale_id	product_id	sale_date	revenue	MAX(revenue) OVER (PARTITION BY product_id)
1	101	2024-01-01	100	200
2	101	2024-01-02	150	200
3	101	2024-01-03	200	200
4	102	2024-01-01	300	400
5	102	2024-01-02	250	400
6	102	2024-01-03	400	400

Using JOINS-

```

3 • SELECT product_id, MAX(revenue)
4   FROM sales
5   group by product_id;

```

Result Grid			Filter Rows:	Export:
	product_id	MAX(revenue)		
▶	101	200		
	102	400		

In this method, the columns in where should match columns in subquery select otherwise it'll throw error-

```

3 • select * from sales s
4   where (s.product_id, s.revenue) in (
5     SELECT product_id, MAX(revenue)
6     FROM sales
7     group by product_id
8   );

```

Result Grid					Filter Rows:	Export:
	sale_id	product_id	sale_date	revenue		
	3	101	2024-01-03	200		
	6	102	2024-01-03	400		

3. Show previous sale revenue for each product.

SELECT *,

LAG(revenue) OVER (PARTITION BY product_id ORDER BY sale_date)

FROM sales;

🧠 Why?

- “Previous” only makes sense with **order**
- Otherwise SQL doesn't know what “previous” means

```

3 • select *, lag(revenue) over (partition by product_id)
4   from sales;

```

	sale_id	product_id	sale_date	revenue	lag(revenue) over (partition by product_id)
▶	1	101	2024-01-01	100	NULL
	2	101	2024-01-02	150	100
	3	101	2024-01-03	200	150
	4	102	2024-01-01	300	NULL
	5	102	2024-01-02	250	300
	6	102	2024-01-03	400	250

4. Identify the day each product had its highest revenue.

```
SELECT sale_id, product_id, sale_date, revenue
```

```
FROM (
```

```
    SELECT *, MAX(revenue) OVER (PARTITION BY product_id) AS max_revenue
```

```
    FROM sales
```

```
) t
```

```
WHERE revenue = t.max_revenue;
```

```

3 • select sale_id, product_id, sale_date, revenue
4   from (select *, max(revenue) over(partition by product_id) as max_revenue
5         from sales) t
6   where revenue = t.max_revenue;

```

	sale_id	product_id	sale_date	revenue
▶	3	101	2024-01-03	200
	6	102	2024-01-03	400

? Your doubt (VERY IMPORTANT)

You asked:

If window functions happen before WHERE, why is this wrong?

```
SELECT sale_id, product_id, sale_date,
```

```
    MAX(revenue) OVER (PARTITION BY product_id) AS max_revenue
```

```
FROM sales
```

```
WHERE revenue = max_revenue;
```

✗ Why this fails

👉 Because of **SQL evaluation order vs alias visibility**

Actual flow:

1. FROM
2. WHERE !
3. WINDOW FUNCTIONS
4. SELECT (alias created here)

🚫 **Problem:**

- `max_revenue` alias is created in **SELECT**
- But **WHERE** runs **before SELECT**
- So **WHERE** cannot see `max_revenue`

🔥 Key Rule (very important)

👉 You cannot use window function results in WHERE directly

✅ Solutions

✓ Subquery (what you did)

✓ OR CTE:

WITH t AS (

SELECT *, MAX(revenue) OVER (PARTITION BY product_id) AS max_revenue

FROM sales

)

SELECT *

FROM t

WHERE revenue = max_revenue;

5. Compare each sale revenue with the average revenue of that product.

SELECT *, revenue - AVG(revenue) OVER (PARTITION BY product_id) AS diff

FROM sales;

```
3 • select *, revenue - avg(revenue) over(partition by product_id) as diff
4   from sales;
```

Result Grid					
Filter Rows:					
Export:  Wrap Cell Content: 					
	sale_id	product_id	sale_date	revenue	diff
▶	1	101	2024-01-01	100	-50.0000
	2	101	2024-01-02	150	0.0000
	3	101	2024-01-03	200	50.0000
	4	102	2024-01-01	300	-16.6667
	5	102	2024-01-02	250	-66.6667
	6	102	2024-01-03	400	83.3333

SCENARIO 4: Daily Website Traffic

```
CREATE TABLE website_visits (  
    visit_date DATE,  
    user_id INT,  
    page_views INT  
);
```

```
INSERT INTO website_visits VALUES  
(  
'2024-01-01', 1, 5),  
(  
'2024-01-01', 2, 3),  
(  
'2024-01-02', 1, 7),  
(  
'2024-01-02', 3, 4),  
(  
'2024-01-03', 2, 6),  
(  
'2024-01-03', 3, 2),  
(  
'2024-01-04', 1, 8),  
(  
'2024-01-04', 2, 1);
```

Result Grid			
Filter Rows:			
	visit_date	user_id	page_views
▶	2024-01-01	1	5
	2024-01-01	2	3
	2024-01-02	1	7
	2024-01-02	3	4
	2024-01-03	2	6
	2024-01-03	3	2
	2024-01-04	1	8
	2024-01-04	2	1

Questions

1. Show total page views per day for each row.

```
SELECT *, SUM(page_views) OVER (PARTITION BY visit_date)  
  
FROM website_visits;
```

```

1 • select *, sum(page_views) over(partition by visit_date)
2   from website_visits;
3

```

Result Grid				
Filter Rows:		Export:  Wrap Cell Content: 		
	visit_date	user_id	page_views	sum(page_views) over(partition by visit_date)
▶	2024-01-01	1	5	8
	2024-01-01	2	3	8
	2024-01-02	1	7	11
	2024-01-02	3	4	11
	2024-01-03	2	6	8
	2024-01-03	3	2	8
	2024-01-04	1	8	9
	2024-01-04	2	1	9

2. Show cumulative page views over time (by date).

SELECT *,

SUM(page_views) OVER (

ORDER BY visit_date

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

)

FROM website_visits;

⚠ **Logically slightly off**

👉 Problem:

- Multiple rows per date → cumulative will increase *within same date*

✅ **Correct approach (aggregate first)**

SELECT visit_date,

SUM(page_views) AS daily_views,

SUM(SUM(page_views)) OVER (

ORDER BY visit_date

ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

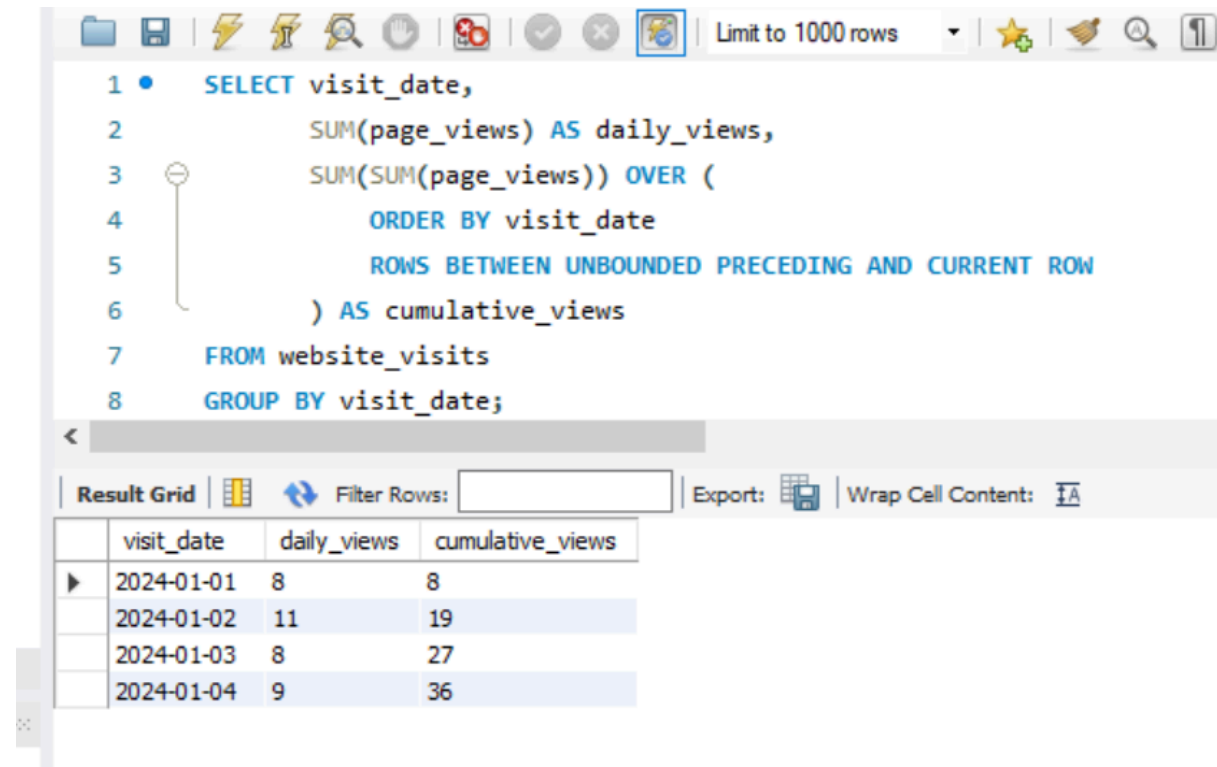
) AS cumulative_views

FROM website_visits

GROUP BY visit_date;

🧠 Key Insight:

👉 Always **aggregate** → **then window** when working with “per day” logic



The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1 • SELECT visit_date,  
2         SUM(page_views) AS daily_views,  
3         SUM(SUM(page_views)) OVER (  
4             ORDER BY visit_date  
5             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
6         ) AS cumulative_views  
7 FROM website_visits  
8 GROUP BY visit_date;
```

Below the query editor is a "Result Grid" section. It includes a "Filter Rows:" input field, an "Export:" button, and a "Wrap Cell Content:" checkbox. The result grid displays the following data:

	visit_date	daily_views	cumulative_views
▶	2024-01-01	8	8
	2024-01-02	11	19
	2024-01-03	8	27
	2024-01-04	9	36

```

1 • select *, sum(page_views) over(order by visit_date
2   rows between unbounded preceding and current row)
3   from website_visits;
4

```

Result Grid Filter Rows: Export: Wrap Cell Content				
	visit_date	user_id	page_views	sum(page_views) over(order by visit_date rows between unbounded preceding and current row)
▶	2024-01-01	1	5	5
	2024-01-01	2	3	8
	2024-01-02	1	7	15
	2024-01-02	3	4	19
	2024-01-03	2	6	25
	2024-01-03	3	2	27
	2024-01-04	1	8	35
	2024-01-04	2	1	36

Limit to 1000 rows				
<pre> 1 • select *, sum(page_views) over(partition by visit_date order by visit_date 2 rows between unbounded preceding and current row) 3 from website_visits; 4 </pre>				
Result Grid Filter Rows: Export: Wrap Cell Content:				
	visit_date	user_id	page_views	sum(page_views) over(partition by visit_date order by visit_date rows between unbounded preceding and current row)
▶	2024-01-01	1	5	5
	2024-01-01	2	3	8
	2024-01-02	1	7	7
	2024-01-02	3	4	11
	2024-01-03	2	6	6
	2024-01-03	3	2	8
	2024-01-04	1	8	8
	2024-01-04	2	1	9

Result 19 ×

3. Show previous day's page views.

```

SELECT visit_date,

       daily_views,

       LAG(daily_views) OVER (ORDER BY visit_date) AS prev_day_views

FROM (

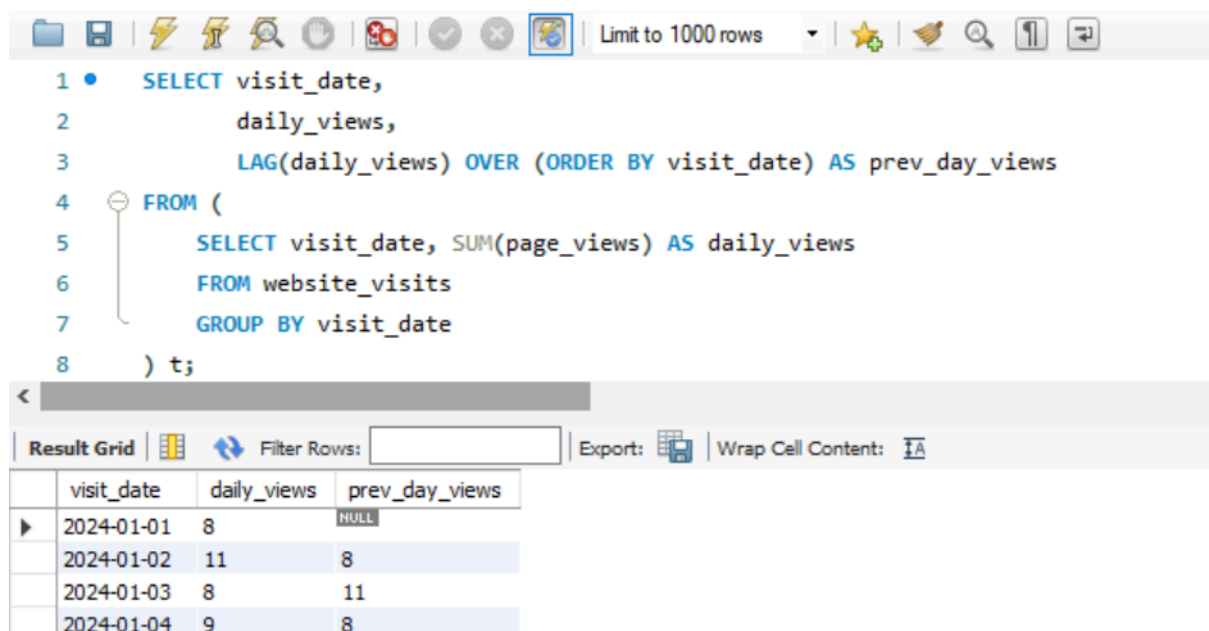
  SELECT visit_date, SUM(page_views) AS daily_views

  FROM website_visits

  GROUP BY visit_date

) t;

```



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the query from the previous block. Below the editor, the 'Result Grid' tab is active, displaying the query results in a table format.

visit_date	daily_views	prev_day_views
2024-01-01	8	NULL
2024-01-02	11	8
2024-01-03	8	11
2024-01-04	9	8

⚠ Q3 — Previous Day Page Views

<> SQL



```

SELECT *,
       LAG(page_views) OVER (ORDER BY visit_date
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
FROM website_visits;

```

✗ Issues:

1. Same problem → multiple users per day
2. Frame clause is unnecessary for LAG()



```

1 • select *, lag(page_views) over(order by visit_date
2   rows between unbounded preceding and current row)
3   from website_visits
4   order by visit_date;
5

```

Result Grid Filter Rows: Export: Wrap Cell Content:				
	visit_date	user_id	page_views	lag(page_views) over(order by visit_date rows between unbounded preceding and current row)
▶	2024-01-01	1	5	NULL
	2024-01-01	2	3	5
	2024-01-02	1	7	3
	2024-01-02	3	4	7
	2024-01-03	2	6	4
	2024-01-03	3	2	6
	2024-01-04	1	8	2
	2024-01-04	2	1	8

4. Show difference between current day and previous day page views.

```

SELECT *,
    current_day_views - LAG(current_day_views) OVER (ORDER BY visit_date) AS diff
FROM (
    SELECT visit_date, SUM(page_views) AS current_day_views
    FROM website_visits
    GROUP BY visit_date
) w1;

```

```

1 • select *, page_views - lag(page_views) over(order by visit_date)
2   from website_visits
3   order by visit_date;
4

```

<				
Result Grid  Filter Rows: Export:  Wrap Cell Content: 				
	visit_date	user_id	page_views	page_views - lag(page_views) over(order by visit_date)
	2024-01-01	1	5	NULL
	2024-01-01	2	3	-2
	2024-01-02	1	7	4
	2024-01-02	3	4	-3
	2024-01-03	2	6	2
	2024-01-03	3	2	-4
	2024-01-04	1	8	6
▶	2024-01-04	2	1	-7

```

9 • select w2.visit_date, sum(w2.page_views)
10   from website_visits w2
11   group by w2.visit_date;

```

<		
Result Grid  Filter Rows: Export:  Wrap		
	visit_date	sum(w2.page_views)
▶	2024-01-01	8
	2024-01-02	11
	2024-01-03	8
	2024-01-04	9


```

2 • select *, w1.current_day_views - lag(w1.current_day_views) over (order by visit_date)
3   from
4   (select w2.visit_date, sum(w2.page_views) as current_day_views
5    from website_visits w2
6    group by w2.visit_date) w1;

```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	visit_date	current_day_views	w1.current_day_views - lag(w1.current_day_views) over (order by visit_date)
▶	2024-01-01	8	NULL
	2024-01-02	11	3
	2024-01-03	8	-3
	2024-01-04	9	1

5. Identify days where traffic increased compared to previous day.

✓ Option 1: Subquery (best)

SELECT *

FROM (

SELECT visit_date,

current_day_views,

current_day_views - LAG(current_day_views) OVER (ORDER BY visit_date) AS diff

FROM (

SELECT visit_date, SUM(page_views) AS current_day_views

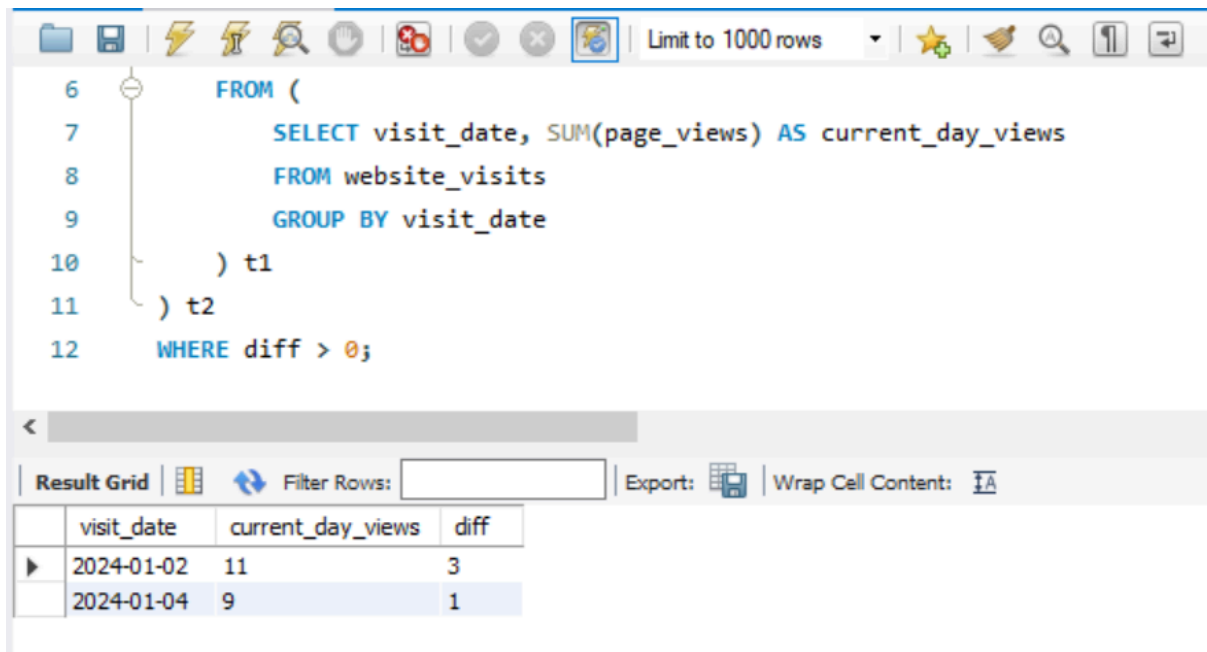
FROM website_visits

GROUP BY visit_date

) t1

) t2

WHERE diff > 0;



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
6 FROM (  
7     SELECT visit_date, SUM(page_views) AS current_day_views  
8     FROM website_visits  
9     GROUP BY visit_date  
10 ) t1  
11 ) t2  
12 WHERE diff > 0;
```

Below the editor is a 'Result Grid' tab with a 'Filter Rows' input and 'Export' and 'Wrap Cell Content' options. The results table has four columns: visit_date, current_day_views, and diff. Two rows are displayed:

visit_date	current_day_views	diff
2024-01-02	11	3
2024-01-04	9	1

✓ Option 2: Cleaner (no diff column)

SELECT *

FROM (

SELECT visit_date,

current_day_views,

LAG(current_day_views) OVER (ORDER BY visit_date) AS prev_day

FROM (

SELECT visit_date, SUM(page_views) AS current_day_views

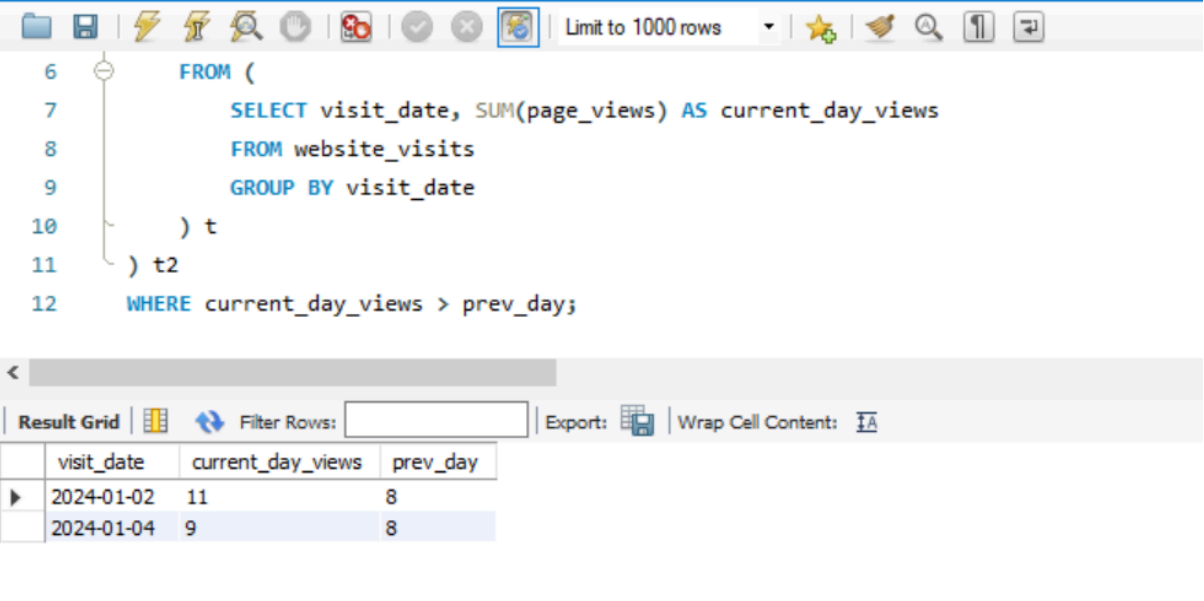
FROM website_visits

GROUP BY visit_date

) t

) t2

WHERE current_day_views > prev_day;



The screenshot shows a SQL IDE interface. The top toolbar includes icons for file operations, execution, and a 'Limit to 1000 rows' dropdown. The SQL editor contains the following query:

```
6 FROM (  
7     SELECT visit_date, SUM(page_views) AS current_day_views  
8     FROM website_visits  
9     GROUP BY visit_date  
10 ) t  
11 ) t2  
12 WHERE current_day_views > prev_day;
```

Below the editor is a 'Result Grid' tab. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The grid displays the following data:

	visit_date	current_day_views	prev_day
▶	2024-01-02	11	8
	2024-01-04	9	8

Important Concepts You Just Learned

1. Aggregate before window (when needed)

Multiple rows per date → GROUP BY first

2. Window functions ≠ usable in WHERE

3. LAG does NOT need frame clause

Only ORDER BY matters